

CSC 583 Project Proposal: Query-Adaptive Retrieval and Ranking of In-Context Examples for LLM Inference

Tian Tan, Linan Zheng, Xinyu Guo
{tiantan, linanzheng, xinyuguo}@arizona.edu

May 7, 2026

Code is available at: <https://github.com/Linan2018/CS583-query-adaptive-ICL>

1 Background and Motivation

In-context learning (ICL) is a capability of large language models (LLM) that allows them to learn and perform tasks without any updates to their underlying parameters. Instead of traditional training, the model learns a task by being shown examples of that task within the same prompt or conversation. Recent advancements in large language models have demonstrated the effectiveness of in-context learning, where models improve their responses by conditioning on examples provided within their context window. While increasing the number of examples often improves performance, studies[8] show that not all examples contribute equally—some are significantly more influential than others. Existing methods typically select a single example set for all queries, assuming a globally optimal combination exists. However, queries may be very different, such as coming from different domains, so a one-size-fits-all approach is inefficient. Prior work, such as BRIDGE[7], attempts to optimize example selection, but it does not dynamically tailor examples to individual queries. As a result, fixed selection strategies may fail to provide the most informative examples for each query, limiting the effectiveness of ICL.

Unlike pre-training and fine-tuning, which require extensive training time and computational resources, in-context learning enhances model performance without modifying its parameters. However, its success depends not just on the number of examples, but on selecting high-quality, contextually relevant ones. A fixed example set may contain irrelevant or redundant examples for certain queries. To fully exploit ICL’s potential, we propose a **query-adaptive example selection** strategy that **dynamically retrieves the most relevant examples for each query**. Unlike previous methods, our approach ensures that each query receives tailored contextual support, improving model accuracy and efficiency. By selecting fewer but better examples, this method enhances ICL’s effectiveness while reducing unnecessary computational overhead, making it a more scalable solution for real-world applications.

When retrieving in-context examples, relying solely on semantic similarity to the query is not the most effective approach. This is because the selection of in-context examples should prioritize task-level similarity, which may diverge from semantic similarity.

For instance, consider a user query such as:

Write down Python code to compute the sum of the first 10 integers.

An example with high semantic similarity but limited utility for the task might be:

Demonstration input: What is the sum of the first 10 integers?

Demonstration output: 55.

Although the example shares semantic content with the query, it does not align well with the task of code generation. The user’s query explicitly requests the model to generate executable Python code, whereas the retrieved example only involves numerical reasoning. Therefore, instead of treating example selection as a semantic similarity problem, we propose to frame it as a recommendation problem—recommending examples that are truly beneficial for solving the specific task at hand, rather than merely semantically similar.

2 Problem Definition

In-context learning performance heavily depends on the quality and relevance of selected examples. Existing selection methods often overlook the fact that different queries may require different demonstrations. Our project aims to address this limitation by developing an adaptive example selection strategy that dynamically selects the most useful examples for each individual query.

We formulate the problem as **query-adaptive example selection**. Let $\mathcal{Q} = \{q_i\}_{i=1}^M$ denote a set of queries and let $\mathcal{C} = \{c_\ell = (x_\ell, y_\ell)\}_{\ell=1}^N$ denote a pool of candidate in-context examples, where each example c_ℓ consists of an input x_ℓ and its corresponding answer y_ℓ . For each query $q_i \in \mathcal{Q}$, our goal is to retrieve a query-specific demonstration set $\mathcal{G}_{q_i} \subseteq \mathcal{C}$ that maximizes the likelihood that the language model produces the correct answer for q_i . Unlike previous approaches that select a fixed demonstration set for all queries, our method optimizes context relevance on a per-query basis, improving the efficiency and adaptability of in-context learning.

3 Proposed Solution

3.1 Intuition

To dynamically retrieve the most relevant examples for each input query, a naive approach is to represent both the examples and queries as vector embeddings using a text embedding model. The similarity between these embeddings, such as cosine similarity, can then be used to identify the most suitable set of examples for each query. This method leverages semantic proximity to ensure that selected examples are contextually relevant and informative. However, research[4] has shown that in some cases, even when embeddings exhibit high similarity, their actual semantic meanings and task requirements can be completely different. Therefore, our proposed approach must address several key **challenges**:

1. **Unreliability of Embedding Similarity.** Relying solely on semantic embedding similarity to identify the most suitable example set is insufficient. Our method must incorporate additional factors to ensure that selected examples align with the true intent and requirements of the query.
2. **Exploding Search Space for Example Selection.** The number of possible example combinations grows exponentially, as N different examples can form 2^N unique subsets. Exhaustively searching all possible combinations for each query is computationally infeasible. To overcome this, we need an efficient and effective sampling strategy to identify the optimal example set.

3.2 Basic Assumptions

Based on the following assumptions, we propose a concrete approach to query-adaptive example selection:

1. **Example usefulness over similarity.** The quality of an example should be judged by how much it helps the language model answer the query, rather than relying solely on semantic similarity.
2. **Query-example relevance within tasks.** In the same task domain, most queries are naturally related to the available examples, meaning effective examples can often be found through simple sampling methods, such as random sampling.
3. **Example selection as recommendation.** Finding the optimal example set for a query can be framed as a recommendation problem, where the goal is to efficiently recommend the most relevant examples rather than enumerating all possible combinations.

3.3 Methodology

3.3.1 Interaction Matrix Construction

Inspired by matrix factorization techniques widely used in recommendation systems, we formulate example selection as a query-specific recommendation problem. Specifically, our goal is to recommend the most suitable demonstration set for each query.

Let $\mathcal{Q}_{\text{train}} = \{q_i\}_{i=1}^M$ denote the set of training queries, and let $\mathcal{C} = \{c_\ell = (x_\ell, y_\ell)\}_{\ell=1}^N$ denote the candidate example pool, where each candidate example consists of an input x_ℓ and its corresponding answer y_ℓ . We sample n demonstration sets from $\mathcal{C}$ and denote them as $\mathcal{G} = \{\mathcal{G}_j\}_{j=1}^n$, where each $\mathcal{G}_j \subseteq \mathcal{C}$ contains one or more candidate examples.

For a training query q_i and a demonstration set $\mathcal{G}_j$, we define the interaction label A_{ij} as:

$$A_{ij} = \begin{cases} 1, & \text{if the LLM correctly answers } q_i \text{ when conditioned on } \mathcal{G}_j, \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

This yields a query-example-set interaction matrix $\mathbf{A} \in \{0, 1\}^{M \times n}$, where each row corresponds to a training query and each column corresponds to a sampled demonstration set.

For example, Table 1 shows an interaction matrix with three queries q_1, q_2 , and q_3 , and three sampled demonstration sets: $\mathcal{G}_1 = \{c_1, c_2\}$, $\mathcal{G}_2 = \{c_2, c_3, c_4\}$, and $\mathcal{G}_3 = \{c_8\}$. The value $A_{11} = 1$ indicates that the language model correctly answers q_1 when conditioned on the demonstration set $\mathcal{G}_1 = \{c_1, c_2\}$.

	$\mathcal{G}_1 = \{c_1, c_2\}$	$\mathcal{G}_2 = \{c_2, c_3, c_4\}$	$\mathcal{G}_3 = \{c_8\}$
q_1	1	0	1
q_2	0	0	0
q_3	1	1	0

Table 1: Example of a query-example-set interaction matrix.

To construct the interaction matrix, we first create a reliable candidate example pool by evaluating each training sample $(x, y) \in \mathcal{D}_{\text{train}}$ under a zero-shot setting. A sample is included in the candidate pool only if the LLM consistently produces the correct answer, ensuring that the demonstrations used in later stages are reliably understood by the model.

Since the number of possible demonstration sets grows exponentially with the number of candidate examples, it is infeasible to evaluate all subsets of $\mathcal{C}$. We therefore sample $n = 20$ demonstration sets from the candidate pool, where each set contains between one and three examples.

We then evaluate each training query q_i under the context of each sampled demonstration set $\mathcal{G}_j$. To mitigate the inherent variability of LLM outputs, we perform five independent inference rounds for each pair $(q_i, \mathcal{G}_j)$ and apply majority voting to determine the final interaction label A_{ij} .

To prevent answer leakage, we exclude any pair (i, j) where the demonstration set $\mathcal{G}_j$ contains the same underlying sample as the query q_i . These excluded entries are masked and are not used for training the factorization model in the following steps.

3.3.2 Matrix Factorization

Given the interaction matrix $\mathbf{A} \in \{0, 1\}^{M \times n}$, we learn a latent representation for each sampled demonstration set. Let $f_{\text{BERT}}(\cdot)$ denote the BERT encoder. For each training query q_i , we compute its embedding as:

$$\mathbf{q}_i = f_{\text{BERT}}(q_i) \in \mathbb{R}^d. \quad (2)$$

Stacking all training query embeddings gives the query embedding matrix:

$$\mathbf{Q}_{\text{train}} = \begin{bmatrix} \mathbf{q}_1^\top \\ \mathbf{q}_2^\top \\ \vdots \\ \mathbf{q}_M^\top \end{bmatrix} \in \mathbb{R}^{M \times d}. \quad (3)$$

We denote the latent embedding matrix of the sampled demonstration sets as:

$$\mathbf{Z} = \begin{bmatrix} \mathbf{z}_1^\top \\ \mathbf{z}_2^\top \\ \vdots \\ \mathbf{z}_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d}, \quad (4)$$

where $\mathbf{z}_j \in \mathbb{R}^d$ is the learnable latent embedding of the demonstration set $\mathcal{G}_j$.

For each query-demonstration-set pair $(q_i, \mathcal{G}_j)$, we compute a compatibility score using the inner product between their embeddings:

$$s_{ij} = \langle \mathbf{q}_i, \mathbf{z}_j \rangle = \mathbf{q}_i^\top \mathbf{z}_j. \quad (5)$$

We then apply a sigmoid function to convert this score into a predicted interaction probability:

$$\hat{A}_{ij} = \sigma(s_{ij}) = \sigma(\langle \mathbf{q}_i, \mathbf{z}_j \rangle) = \frac{1}{1 + \exp(-\langle \mathbf{q}_i, \mathbf{z}_j \rangle)}. \quad (6)$$

In matrix form, the predicted interaction matrix is:

$$\hat{\mathbf{A}} = \sigma(\mathbf{Q}_{\text{train}} \mathbf{Z}^\top), \quad (7)$$

where $\sigma(\cdot)$ is applied element-wise.

Let Ω denote the set of observed and unmasked entries in the interaction matrix. We define the positive and negative interaction sets as:

$$\Omega^+ = \{(i, j) \in \Omega \mid A_{ij} = 1\}, \quad \Omega^- = \{(i, j) \in \Omega \mid A_{ij} = 0\}. \quad (8)$$

During training, we construct a sampled training set $\mathcal{S} = \mathcal{S}^+ \cup \mathcal{S}^-$, where $\mathcal{S}^+ \subseteq \Omega^+$ and $\mathcal{S}^- \subseteq \Omega^-$. In practice, we sample positive and negative instances with a ratio of 1 : 1.

We optimize the latent demonstration-set embeddings by minimizing the binary cross-entropy loss:

$$\mathcal{L} = -\frac{1}{|\mathcal{S}|} \sum_{(i,j) \in \mathcal{S}} \left[A_{ij} \log \hat{A}_{ij} + (1 - A_{ij}) \log(1 - \hat{A}_{ij}) \right]. \quad (9)$$

To address class imbalance, we also consider a weighted binary cross-entropy loss:

$$\mathcal{L}_w = -\frac{1}{|\mathcal{S}|} \sum_{(i,j) \in \mathcal{S}} \left[w_+ A_{ij} \log \hat{A}_{ij} + w_- (1 - A_{ij}) \log(1 - \hat{A}_{ij}) \right], \quad (10)$$

where w_+ and w_- are the weights for positive and negative instances, respectively. In practice, the sparsity of positive interactions is around 40%, so we set $w_+ = 0.6$ and $w_- = 0.4$ to balance the contribution of each class.

By minimizing these objectives, the model learns latent representations of demonstration sets that assign higher scores to effective query-demonstration pairs and lower scores to ineffective ones.

3.3.3 Inference

After training, we use the learned latent matrix $\mathbf{Z}$ to recommend demonstration sets for unseen test queries. Given a test query q_* , we first compute its BERT embedding:

$$\mathbf{q}_* = f_{\text{BERT}}(q_*) \in \mathbb{R}^d. \quad (11)$$

We then compute the predicted interaction probability between q_* and each sampled demonstration set $\mathcal{G}_j$:

$$\hat{A}_{*,j} = \sigma(\langle \mathbf{q}_*, \mathbf{z}_j \rangle), \quad j = 1, \dots, n. \quad (12)$$

The best demonstration set is selected as:

$$j^* = \arg \max_{j \in \{1, \dots, n\}} \hat{A}_{*,j}, \quad \mathcal{G}_{q_*} = \mathcal{G}_{j^*}. \quad (13)$$

For the top- k strategy used in our experiments, we select the indices of the k highest-scoring demonstration sets:

$$\mathcal{J}_k(q_*) = \text{TopK}_{j \in \{1, \dots, n\}}(\hat{A}_{*,j}), \quad (14)$$

and use the corresponding demonstration sets $\{\mathcal{G}_j \mid j \in \mathcal{J}_k(q_*)\}$ for inference.

4 What We Have Done

Overall, we have implemented the method proposed in the early stage of the project and conducted a series of insightful experiments, including performance-oriented optimizations and data processing strategies. In the following sections, we describe the experimental setup we established, the results obtained so far, and the various efforts made during the optimization process. The experiments were designed to address the following objectives:

- **Performance Comparison:** First, we aimed to evaluate how our proposed method performs relative to existing approaches.
- **Statistical Analysis:** We conducted a series of statistical analyses, including estimating the upper bound of test accuracy, evaluating the performance of the trained model, and analyzing metrics such as precision and recall for the recommended example combinations.
- **Ablation Studies:** Finally, we performed a set of ablation studies to assess the effectiveness of various strategies, such as handling class imbalance, mitigating LLM output instability, and improving example recommendation strategies.

4.1 Basic Setting

Our experiments are conducted using Gemini 1.5 Pro (gemini-1.5-pro-001) [6] as the backbone large language model and focus on the BIG-Bench Hard (BBH) [5] benchmark, a collection of 26 challenging reasoning tasks that span numerical reasoning, commonsense problem-solving, logical deduction, and tabular inference. In our initial study, we focus on the Causal Judgement task. After randomly shuffling the dataset, we split it into 65% training and 35% testing, with all adaptive example selection and learning conducted exclusively on the training partition. We use a standardized prompting format inspired by prior work, with a Jinja2-style template as follows:

```
You will be given a question. Think step by step before giving a final answer to this
question. Show your final answer {{ TASK_SPECIFIC_CONSTRAINTS }} between <answer>
and </answer>

{{ EXAMPLES }}
==

{{ QUESTION }}
{{ llm() }}
```

The upper-cased blocks bracketed between double braces are variables that are replaced at inference time: `TASK_SPECIFIC_CONSTRAINTS` denote the constraint instruction specific to the type of the task. `EXAMPLES` denote the concatenation of any examples e added to the input. Finally, `llm()` denotes the place where an LLM response is solicited; the answer is then extracted and postprocessed to match with a ground-truth answer to measure accuracy.

4.2 Evaluation Metrics

To comprehensively evaluate the effectiveness of our query-adaptive example selection strategy, we adopt the following metrics:

Accuracy. The primary evaluation metric is the proportion of test queries for which the LLM produces a correct answer when augmented with the recommended demonstration set. Formally, given a test set $\mathcal{Q}_{\text{test}}$, accuracy is defined as:

$$\text{Accuracy} = \frac{|\{q \in \mathcal{Q}_{\text{test}} \mid \hat{r}_q = r_q\}|}{|\mathcal{Q}_{\text{test}}|}, \quad (15)$$

where $\hat{r}_q$ denotes the LLM’s predicted answer for query q and r_q is the corresponding ground-truth answer.

Majority Voting Accuracy. To mitigate the inherent stochasticity in LLM outputs, we perform $K = 3$ independent inference rounds for each test query and apply majority voting. A query is considered correctly answered if the majority of the K responses match the ground truth. This provides a more robust and stable estimate of model performance compared to single-round evaluation.

Precision and Recall of Recommended Example Sets. Beyond end-task accuracy, we also evaluate the quality of the recommended example combinations. Precision measures the proportion of recommended example sets that actually lead to correct LLM responses, while recall measures the proportion of effective example sets (those leading to correct answers) that are successfully retrieved by our recommendation model. These metrics help assess whether the model can accurately identify beneficial demonstrations while avoiding ineffective ones.

4.3 Experimental Procedure

Our experimental pipeline consists of several stages designed to implement and evaluate the proposed query-adaptive example selection strategy.

Step 1: Construction of a Reliable Candidate Example Pool. We begin by constructing a high-quality candidate example pool. Each training sample (i.e., query) is first evaluated under a zero-shot setting using the LLM. A query is included in the candidate example pool if it consistently receives correct answers across multiple zero-shot inference rounds. This filtering ensures that only queries reliably understood by the model are used as examples in subsequent demonstration generation.

Step 2: Embedding Extraction with BERT. We use the pre-trained BERT-base [1] model to encode both training and testing samples into semantic embeddings. BERT (Bidirectional Encoder Representations from Transformers) is a widely adopted and robust language representation model known for capturing rich contextual semantics. Its effectiveness and generalizability make it a suitable choice for embedding textual data prior to matrix factorization. These embeddings serve as inputs for constructing a joint query-example interaction space.

Step 3: Construction of the Query-Example Interaction Matrix. We randomly sample combinations of candidate examples to form demonstration sets. These sets are then appended as context to each training query and passed to the LLM to assess the model’s performance in the presence of demonstrations. As outlined in Section 3.3, the interaction is labeled 1 if the LLM’s response exactly matches the ground-truth answer, and 0 otherwise. Repeating this process across multiple sampled combinations and queries yields an interaction matrix $\mathbf{A}$, where each entry captures the model’s response correctness for a specific query under a specific example set.

To mitigate the inherent variability in LLM predictions, we perform $K = 3$ independent inference rounds per query-example pair and apply a majority voting scheme to determine the final label. This reduces the impact of stochastic behavior and ensures more reliable measurement of model performance.

Step 4: Matrix Factorization and Example Set Prediction. Given the constructed interaction matrix $\mathbf{A}$, we use the BERT-derived query embedding matrix $\mathbf{Q}_{\text{train}}$ to learn a latent embedding matrix $\mathbf{Z}$ for the sampled demonstration sets. The predicted interaction matrix is modeled as:

$$\hat{\mathbf{A}} = \sigma(\mathbf{Q}_{\text{train}} \mathbf{Z}^T), \quad (16)$$

where $\sigma(\cdot)$ is applied element-wise. We train this model by minimizing the binary cross-entropy loss over sampled positive and negative entries of $\mathbf{A}$.

We train this model using the Adam optimizer [3]. Additionally, to address class imbalance in the training set, we apply a class-weighted loss [2], which reduces the bias toward the majority class and encourages more balanced learning across positive and negative interactions.

Step 5: Example Set Recommendation for Test Queries. After training, we use the learned latent matrix $\mathbf{Z}$ to estimate interaction probabilities for test queries. For each test query q_* , we compute its embedding $\mathbf{q}_*$ and obtain the predicted score vector:

$$\hat{\mathbf{a}}_* = \sigma(\mathbf{Z} \mathbf{q}_*). \quad (17)$$

Each entry $\hat{A}_{*,j}$ represents the predicted probability that demonstration set $\mathcal{G}_j$ will help the LLM correctly answer q_* .

We then select the top- k demonstration sets according to these scores and use them as in-context examples. The LLM’s response is evaluated against the ground truth, and the resulting accuracy is used to measure the effectiveness of the recommended demonstration sets.

Throughout our evaluation, we compare our approach against BRIDGE[7], one of the most recent and effective baseline methods for many-shot in-context learning.

4.4 Current Results

In our experiments, we randomly sampled 20 candidate example combinations from the constructed example pool. After training our matrix factorization model on the training split, we applied the learned embedding matrix to recommend the top-3 most likely effective example sets for each test query.

The latest experimental results are presented in Fig. 1, which compares our method with following baselines reported in [7]:

- Direct: Uses only the query and target (answer) in the demonstrations, without including model-generated rationales.
- Chain-of-Thought (CoT): Prompts the LLM to first generate rationales and answers; the demonstration then includes the full concatenation of query, rationale, and target—regardless of whether the rationale leads to the correct prediction.
- Bridge-1o: A SOTA example selection strategy that uses Bayesian optimization to identify a single globally optimal example set, which is then applied to all test queries.

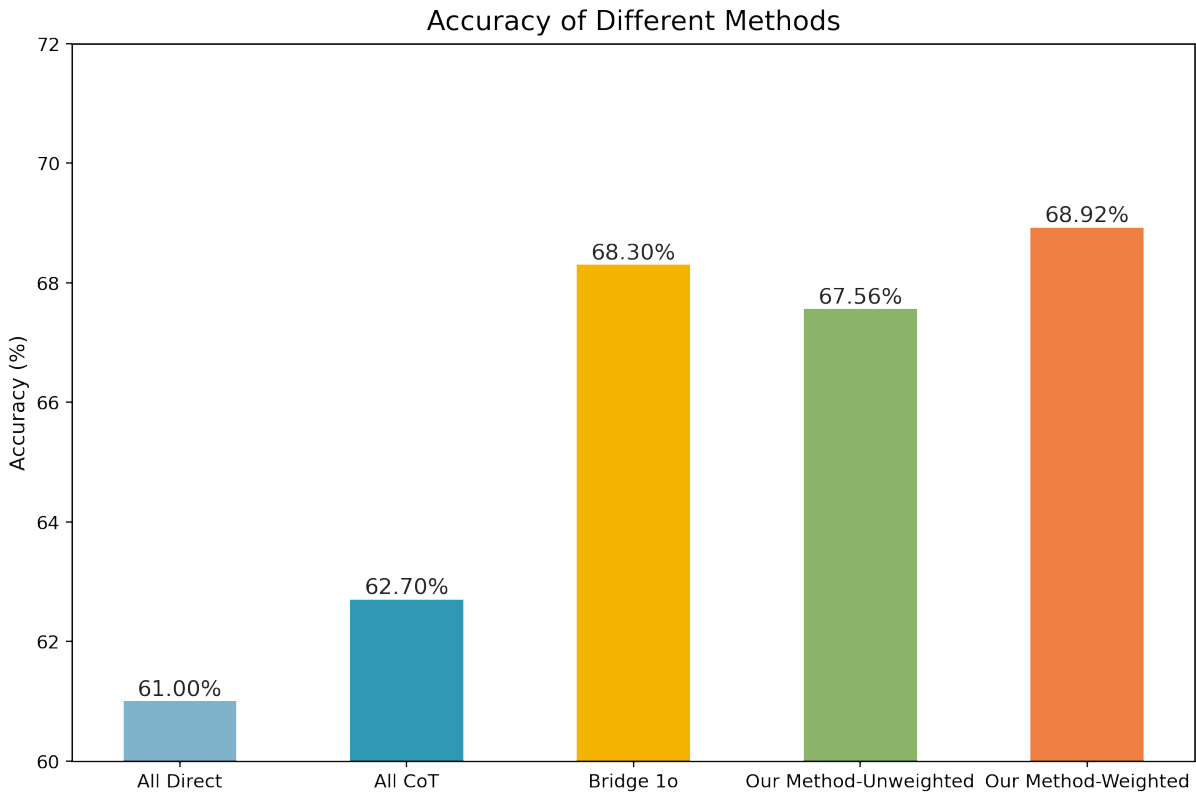


Figure 1: Accuracy comparison of different demonstration strategies.

The last two bars in the figure correspond to our proposed Query-Adaptive Example Selection approach. The Unweighted variant does not apply any loss reweighting to address class imbalance, whereas the Weighted version introduces class-balanced loss weighting to mitigate the overprediction of majority labels.

Notably, our Weighted method achieves accuracy that is comparable to Bridge-1o, despite using only 20 sampled example combinations—significantly reducing computational cost. These results demonstrate the feasibility and potential of our query-adaptive selection strategy, particularly in scenarios where large-scale optimization over all combinations is impractical.

4.4.1 Additional Experiments

During the course of this project, we conducted several additional experiments to improve performance and validate results.

Upper bound of test set accuracy. Furthermore, we estimated the theoretical upper bound of accuracy on the test set, which allowed us to better understand the performance ceiling and how far we are from it. Interestingly, we found that this theoretical upper limit was not 100%. Our analysis suggests two main reasons: (1) some queries in the test set may be inherently difficult or fall into edge cases that the language model cannot handle correctly, regardless of the demonstrations provided; and (2) the current demonstration sampling space may be insufficient — meaning that among the 20 sampled combinations, none are capable of guiding the model to the correct answer for certain queries.

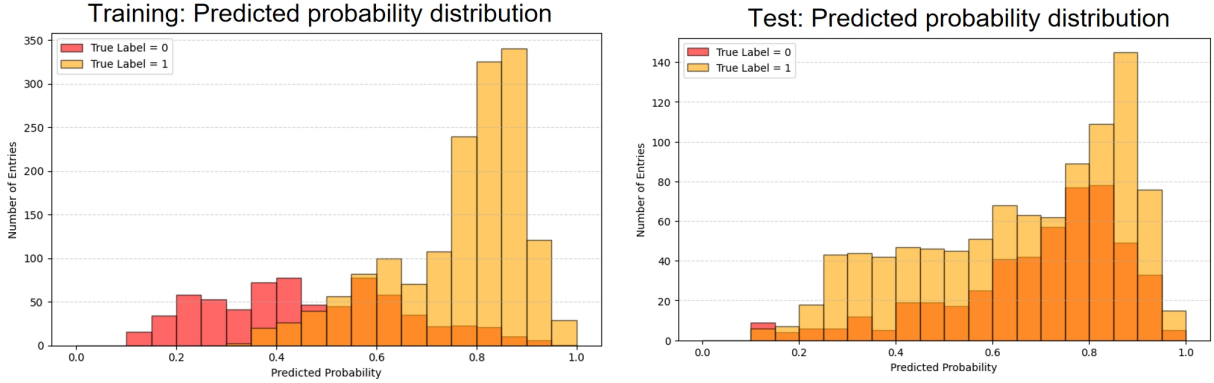


Figure 2: Predicted probability distribution comparison of training & test sets.

Validating trained model performance. In addition, to further assess the capability of the trained model, we used the learned latent embedding matrix $\mathbf{Z}$ to construct query-example-set interaction matrices for both the training and test sets. We then visualized the predicted probability distributions, as shown in Fig. 2.

On the training set, the predicted probabilities align with our expectations: samples with a ground-truth label of 0 tend to receive lower predicted probabilities, while those labeled as 1 tend to receive higher ones. This distinction is clearly visible in the histogram on the left.

However, on the test set, the predicted probabilities for both positive and negative samples follow a very similar distribution, which is undesirable. It suggests that the model fails to clearly distinguish between the two classes, indicating poor generalization ability.

This observation led us to suspect that the training data may suffer from class imbalance. Additionally, it is worth noting that our model might be overly simplistic, with limited capacity to capture and express complex features—another potential cause of this issue. After applying loss weighting, the class imbalance issue was alleviated to some extent, and the test accuracy also improved, although the gain was not particularly significant.

Recommendation strategy optimization. One such effort focused on optimizing the strategy for recommending demonstration combinations to test queries. In the initial design, we simply selected the single highest-scoring example combination for each test query, which yielded an accuracy of around 0.62 to 0.64. We later switched to a top- k strategy, which led to a notable improvement in accuracy. This aligns with the intuition that test queries benefit from having access to more helpful demonstrations.

In addition, we experimented with a hybrid strategy that selects the top- k recommended combinations along with the most common example combination—that is, the one that helps the largest number of test queries produce correct answers. The accuracy results of these strategies are presented in Fig. 3.

An interesting observation is that combining the top-2 highest-scoring example combinations with the most common combination yields lower accuracy than simply using the top-3 highest-scoring combinations. This result is intuitive: recommending a single globally optimal demonstration combination does not necessarily benefit all queries. While it may help a large portion of them, it can be irrelevant—or even detrimental—for certain queries, potentially introducing misleading information. This highlights the importance of query-specific, rather than globally uniform, demonstration selection strategies.

Mitigating LLM Output Instability. To mitigate the issue of LLM output instability, we adopted a multi-round querying strategy combined with majority voting. Specifically, each test query was submitted

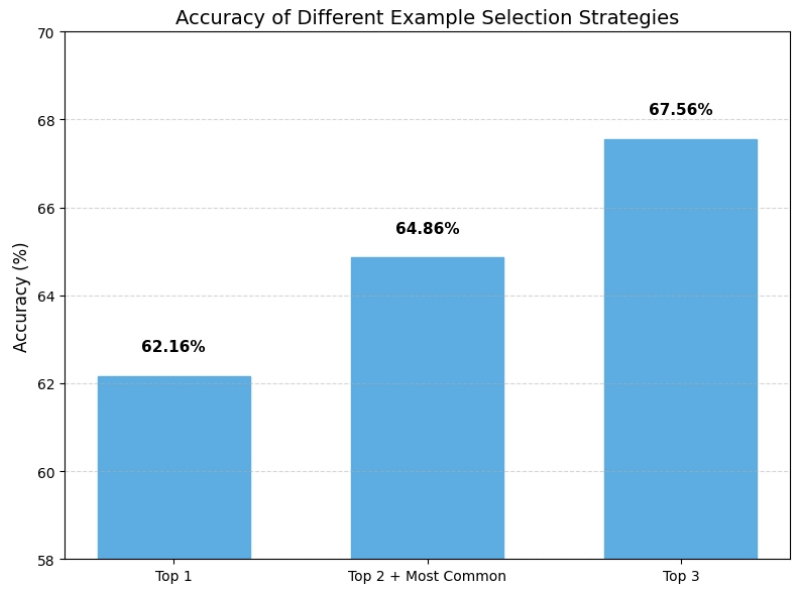


Figure 3: Accuracy of different example recommendation strategies

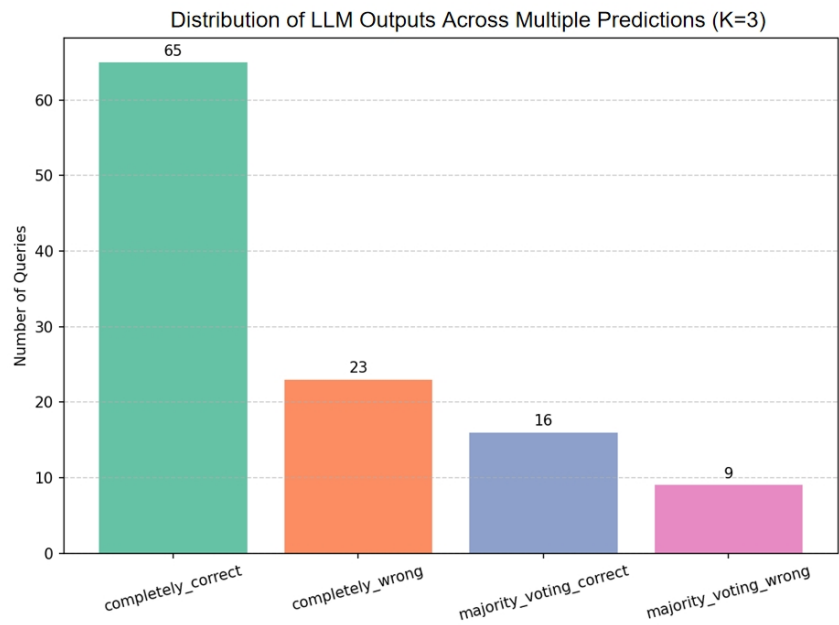


Figure 4: Distribution of LLM Outputs Across Multiple Predictions (K=3)

to the LLM $K=3$ times, and we recorded the correctness of each individual response. A query was labeled as *completely correct* if all three responses were correct, and as *majority voting correct* if the model answered correctly in the majority of cases (e.g., 2 out of 3). Conversely, queries that were incorrectly answered in all or the majority of attempts were labeled as *completely incorrect* or *majority voting incorrect*, respectively.

We considered *majority voting correct* queries to be those that the LLM has the potential to answer correctly, despite occasional randomness or noise in its output. The distribution of these categories is shown in Fig. 4.

From the results, we observe that a considerable proportion of queries fall into the majority voting correct category, indicating that the LLM exhibits some degree of output instability. Nonetheless, most queries are classified as completely correct, suggesting that this noise is generally limited and manageable. In such cases, majority voting proves to be an effective strategy—it helps correct occasional errors and improves overall accuracy. Relying on a single LLM response, by contrast, would lead to greater variability and less reliable results.

5 Future Work

Through this project, we have learned a great deal. Most notably, we successfully built a query-specific demonstration recommendation system. Rather than relying solely on semantic similarity between queries and examples, our system takes into account the actual performance of a large language model (LLM) when different example combinations are provided, enabling it to select the most helpful ones for each query.

We also learned how to interact with LLMs via API calls and how to dynamically construct prompt templates based on different task requirements. More importantly, our extensive experimentation led to a number of insightful observations:

1. **Instability in LLM responses:** We observed that LLM outputs can sometimes be inconsistent. In some cases—especially when the input is long or for unclear reasons—the model would return empty responses. For slightly more challenging queries, the model often gave different answers across multiple runs, suggesting an inherent randomness or uncertainty in its behavior. As a result, when the test set is evaluated multiple times using the same recommended example combinations, the LLM’s accuracy exhibits noticeable fluctuations.
2. **Increasing the number of demonstrations generally helps:** Providing more demonstrations tends to improve LLM performance, as the model has access to more reference information. However, blindly increasing the number of examples is not necessarily beneficial. The key lies in the quality of the examples—only those relevant and helpful to the current query lead to improved accuracy. Low-quality demonstrations may even mislead the model.
3. **Improving model fit to the training data:** The performance of our trained model can be further enhanced by better fitting the query-example interaction matrix on the training set. This can be approached from two directions:
 - Model-side optimization:** This includes designing more expressive model architectures, tuning hyperparameters more rigorously (e.g., via grid search rather than heuristic values), and exploring alternative loss functions.
 - Embedding-side optimization:** Our current query representations are obtained from a BERT encoder, resulting in high-dimensional embeddings. This high dimensionality might hinder the model’s ability to learn effectively. Dimensionality reduction or regularization techniques could be explored to address this issue.

In addition, we plan to apply clustering techniques to better understand the structure and distribution of the dataset. This can help detect domain shift issues and guide the development of more effective example sampling strategies. For instance, clustering examples and then sampling from different clusters may increase the diversity of demonstrations and improve overall performance. Overall, our current experimental results suggest that the proposed method is feasible and holds potential for outperforming existing state-of-the-art methods.

6 Usage of LLMs

We did not use large language models for code generation in this project. After running the experiments, we used LLMs to help organize and refine the experimental code, improving its readability and structure. Overall, we leveraged LLMs in two other aspects of our workflow. First, we used LLMs to assist with polishing and refining the writing of this report, including improving clarity, grammar, and overall coherence. Second, LLMs were used to polish prompt templates used for in-context learning experiments. Specifically, we consulted LLMs to refine the wording of task-specific instructions and to ensure that the prompt format effectively guided the model toward producing well-structured and accurate responses. Third, when encountering difficult-to-detect bugs during implementation, we occasionally used LLMs to assist with debugging and to provide explanatory guidance, in addition to searching for solutions online.

References

- [1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- [2] Haibo He and Eduardo A Garcia. Learning from imbalanced data. *IEEE Transactions on knowledge and data engineering*, 21(9):1263–1284, 2009.
- [3] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [4] Chengwei Qin, Aston Zhang, Chen Chen, Anirudh Dagar, and Wenming Ye. In-context learning with iterative demonstration selection. *arXiv preprint arXiv:2310.09881*, 2023.
- [5] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, , and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*, 2022.
- [6] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.
- [7] Xingchen Wan, Han Zhou, Ruoxi Sun, and Serkan O Arik. From few to many: Enhancing many-shot in-context learning with optimized example selection and expansion. In *The Thirteenth International Conference on Learning Representations*.
- [8] What Makes In-Context Learning Work. Rethinking the role of demonstrations: What makes in-context learning work?